

TRAVAUX PRATIQUES — MICROSERVICES

TP 1

Application Monolithique
avec Spring Boot

Gestion de Produits Simple

 Étudiante : Rayhana Laznasni
 Filière : CSIA — Cybersécurité & Intelligence Artificielle
 Module : Architecture Microservices
 Année : 2024 – 2025
 Technologie : Spring Boot 3 · Java 17 · PostgreSQL

 CSIA — Cybersécurité
& Intelligence Artificielle

Table des matières

1	Introduction	2
1.1	Objectifs pédagogiques	2
1.2	Outils nécessaires	2
2	Architecture de l'Application	2
3	Étapes de Réalisation	3
4	Base de Données H2 — Choix Technique	9
4.1	Pourquoi H2 ?	9
4.2	Configuration — Mode Fichier pour ne pas perdre les données	10
5	Tests Postman — Captures d'écran	11
6	Exercice — Entité Category	17
7	Analyse — Avantages et Limites du Monolithe	18
8	Conclusion	19

1 Introduction

Dans ce TP, nous introduisons le développement d'une **application monolithique** avec **Spring Boot**, en suivant une architecture en couches classique. Avant de passer aux microservices, il est essentiel de maîtriser les fondamentaux d'une API REST complète dans un contexte monolithique.

Une **application monolithique** est une application où toutes les fonctionnalités sont regroupées dans un seul projet déployable. Contrairement aux microservices, toutes les couches (présentation, métier, données) coexistent dans le même processus.

1.1 Objectifs pédagogiques

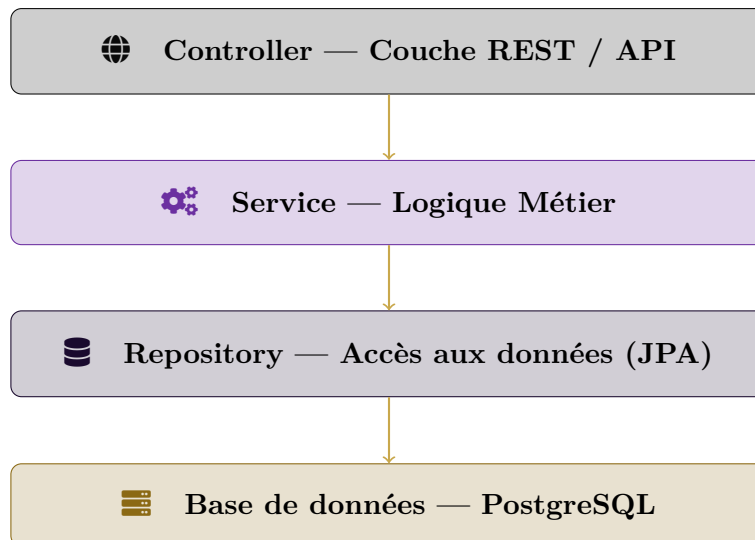
- ✓ Créer une application Spring Boot monolithique simple
- ✓ Développer une API REST CRUD complète
- ✓ Comprendre l'architecture en couches (Entity, Repository, Service, Controller)
- ✓ Découvrir `RestRepository` pour simplifier le code
- ✓ Identifier les avantages et limites d'un monolithe

1.2 Outils nécessaires

Outil	Version / Détail
JDK	17 ou supérieur
Maven	3.8 ou supérieur
IDE	IntelliJ IDEA / Eclipse / VS Code
Base de données	PostgreSQL ou MySQL
Test API	Postman

2 Architecture de l'Application

L'application suit le patron d'architecture en **4 couches** typique de Spring Boot :



3 Étapes de Réalisation



Étape 1 — Initialisation du Projet

Nous avons créé notre projet sur start.spring.io avec les dépendances suivantes :

Dépendance	Rôle
Spring Web	Création des endpoints REST
Spring Data JPA	ORM et accès base de données
PostgreSQL Driver	Connecteur PostgreSQL
Lombok	Réduction du code boilerplate
Validation	Validation des données d'entrée

Configuration `application.properties` :

`application.properties`

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

🏠 Étape 2 — Création de l'Entité Product

L'entité `Product` représente la table `products` en base de données. Nous avons utilisé les annotations Lombok (`@Getter`, `@Setter`, `@Builder`) pour éliminer le code répétitif.

Product.java

```
1 package com.ecommerce.monolith.product.model;
2
3 import jakarta.persistence.*;
4 import jakarta.validation.constraints.*;
5 import lombok.*;
6 import java.math.BigDecimal;
7
8 @Entity
9 @Table(name = "products")
10 @Getter @Setter
11 @NoArgsConstructor @AllArgsConstructor
12 @Builder
13 public class Product {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @NotBlank(message = "Product name is required")
20     @Column(nullable = false)
21     private String name;
22
23     @Column(length = 1000)
24     private String description;
25
26     @NotNull(message = "Price is required")
27     @Positive(message = "Price must be greater than 0")
28     @Column(nullable = false, precision = 10, scale = 2)
29     private BigDecimal price;
30
31     @NotNull(message = "Stock is required")
32     @PositiveOrZero(message = "Stock cannot be negative")
33     @Column(nullable = false)
34     private Integer stock;
35 }
```

⚠ Points importants :

- `@Entity` déclare la classe comme entité JPA
- `@GeneratedValue` gère l'auto-incrément de l'ID
- Les annotations de validation empêchent les données invalides



Étape 3 — Création du Repository

Spring Data JPA nous propose deux approches pour le Repository :

Version Classique

ProductRepository.java — Version classique

```
1 package com.ecommerce.monolith.product.repository;
2
3 import com.ecommerce.monolith.product.model.Product;
4 import
    org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6 import java.util.List;
7
8 @Repository
9 public interface ProductRepository extends
    JpaRepository<Product, Long> {
10     List<Product> findByNameContainingIgnoreCase(String
        name);
11 }
```

Version RestRepository (API automatique)

ProductRepository.java — Version RestRepository

```
1 @RepositoryRestResource(path = "products")
2 public interface ProductRepository extends
    JpaRepository<Product, Long> {
3     List<Product> findByNameContaining(@Param("name")
        String name);
4 }
```



Avantage de @RepositoryRestResource :

Spring génère **automatiquement** tous les endpoints REST sans écrire de Controller ! Idéal pour les prototypes rapides.



Étape 4 — Création du Service

La couche Service contient la **logique métier** et orchestre nos appels au Repository.

ProductService.java

```
1 package com.ecommerce.monolith.product.service;
2
3 import com.ecommerce.monolith.product.model.Product;
4 import
5     com.ecommerce.monolith.product.repository.ProductRepository;
6 import lombok.RequiredArgsConstructor;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9 import java.util.List;
10
11 @Service
12 @RequiredArgsConstructor
13 @Transactional
14 public class ProductService {
15
16     private final ProductRepository repository;
17
18     @Transactional(readOnly = true)
19     public List<Product> getAll() {
20         return repository.findAll();
21     }
22
23     @Transactional(readOnly = true)
24     public Product getById(Long id) {
25         return repository.findById(id)
26             .orElseThrow(() ->
27                 new IllegalArgumentException("Product not
28                     found: " + id));
29     }
30
31     public Product create(Product product) {
32         return repository.save(product);
33     }
34
35     public Product update(Long id, Product details) {
36         Product product = getById(id);
37         product.setName(details.getName());
38         product.setDescription(details.getDescription());
39         product.setPrice(details.getPrice());
40         product.setStock(details.getStock());
41         return product; // save inutile si entite managed
42     }
43
44     public void delete(Long id) {
45         Product product = getById(id);
46         repository.delete(product);
47     }
48 }
```



Étape 5 — Création du Controller

Le Controller expose les endpoints REST et délègue le traitement au Service que nous avons défini.

ProductController.java

```
1 package com.ecommerce.monolith.product.controller;
2
3 import com.ecommerce.monolith.product.model.Product;
4 import
5     com.ecommerce.monolith.product.service.ProductService;
6 import lombok.RequiredArgsConstructor;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.*;
10 import jakarta.validation.Valid;
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/api/products")
15 @RequiredArgsConstructor
16 public class ProductController {
17
18     private final ProductService service;
19
20     @GetMapping
21     public ResponseEntity<List<Product>> getAll() {
22         return ResponseEntity.ok(service.getAll());
23     }
24
25     @GetMapping("/{id}")
26     public ResponseEntity<Product> getById(@PathVariable
27         Long id) {
28         return ResponseEntity.ok(service.getById(id));
29     }
30
31     @PostMapping
32     public ResponseEntity<Product> create(@Valid
33         @RequestBody Product p) {
34         return ResponseEntity.status(HttpStatus.CREATED)
35             .body(service.create(p));
36     }
37
38     @PutMapping("/{id}")
39     public ResponseEntity<Product> update(
40         @PathVariable Long id,
41         @Valid @RequestBody Product p) {
42         return ResponseEntity.ok(service.update(id, p));
43     }
44
45     @DeleteMapping("/{id}")
46     public ResponseEntity<Void> delete(@PathVariable Long
47         id) {
48         service.delete(id);
49         return ResponseEntity.noContent().build();
50     }
51 }
```



Étape 6 — Tests avec Postman

Méthode	URL	Description
GET	/api/products	Récupérer tous les produits
GET	/api/products/{id}	Récupérer un produit par ID
POST	/api/products	Créer un nouveau produit
PUT	/api/products/{id}	Modifier un produit existant
DELETE	/api/products/{id}	Supprimer un produit

Exemple de corps JSON pour POST / PUT :

Corps de la requête — JSON

```
{
  "name": "Laptop Dell XPS",
  "description": "Ordinateur portable haute performance",
  "price": 1299.99,
  "stock": 15
}
```

4 Base de Données H2 — Choix Technique

Lors de ce TP, nous avons rencontré des **problèmes de connexion avec PostgreSQL** (erreurs de driver, configuration de port, authentification). Pour ne pas bloquer l'avancement, nous avons opté pour la base de données embarquée **H2**, qui ne nécessite aucune installation externe.

4.1 Pourquoi H2 ?

Avantage	Détail
Embarquée	Aucune installation requise, intégrée au projet Maven
Zéro configuration	Fonctionne immédiatement avec Spring Boot
Console web	Interface graphique sur /h2-console
Persistance disque	Données sauvegardées sur fichier .mv.db
Compatible JPA	Même usage que PostgreSQL pour les entités

4.2 Configuration — Mode Fichier pour ne pas perdre les données

Sauvegarde automatique sur disque :

Par défaut, H2 fonctionne en mode **mémoire** — toutes les données sont **perdues** au redémarrage. En utilisant le mode **fichier** (`file:`), nous avons configuré la persistance sur disque entre chaque session.

application.properties — H2 persistance sur disque

```
# H2 mode fichier (les donnees sont sauvegardees sur disque)
spring.datasource.url=jdbc:h2:file:./data/ecommercedb;AUTO_SERVER=
TRUE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Console H2 accessible via http://localhost:8080/h2-console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Fichier généré automatiquement :

Un dossier `data/` est créé à la racine du projet avec le fichier `ecommercedb.mv.db`. Ce fichier contient toutes nos données — **aucune perte** même après redémarrage du serveur.

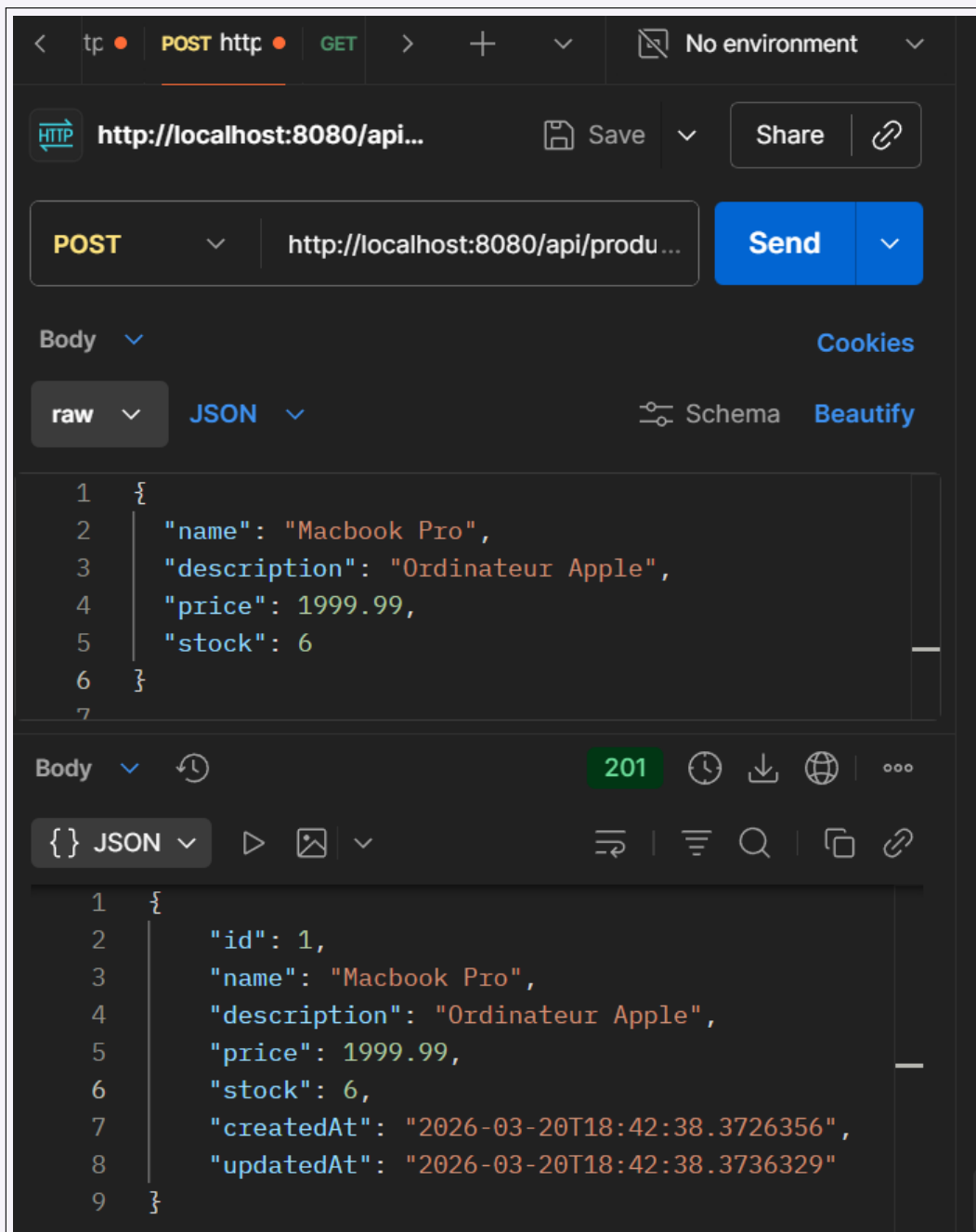
pom.xml — Dépendance H2 à ajouter

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

5 Tests Postman — Captures d'écran

- ✓ URL de base : `http://localhost:8080`
- ✓ Base de données : H2 mode fichier (persistance disque)
- ✓ Format : JSON (Content-Type: `application/json`)
- ✓ Outil : Postman

+ POST — Créer un produit



Insérer ici le screenshot Postman de la requête `POST /api/products` — réponse `201 Created`.

GET — Lister tous les produits

POST http://localhost:8080/api/...

GET http://localhost:8080/api/...

http://localhost:8080/api/...

Save

Share

GET

http://localhost:8080/api/produ...

Send

Params

Cookies

Query Params

	Key	Value	D...	...	Bulk Edit
	Key	Value	Description		

Body

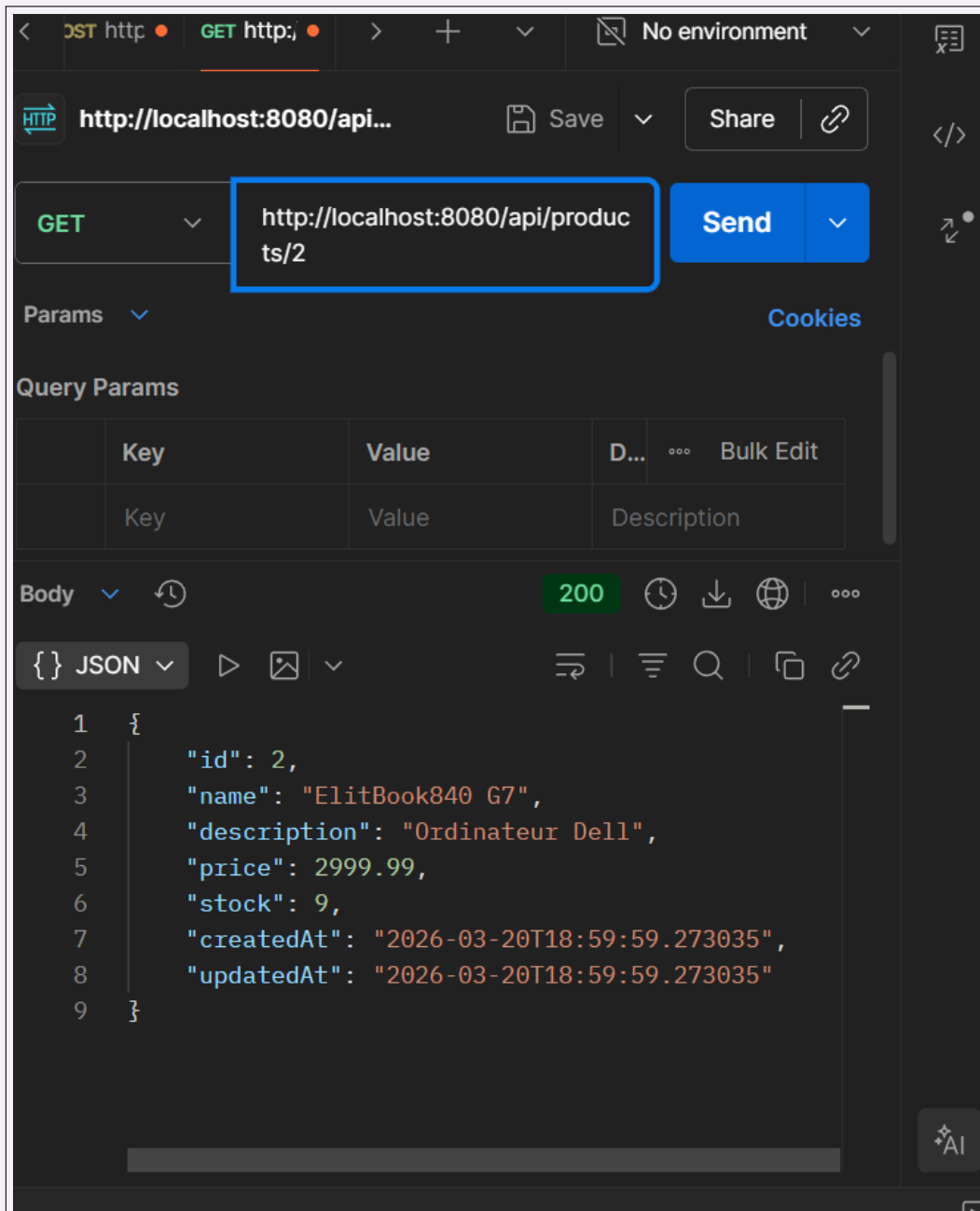
200

{ } JSON

```
1  [
2    {
3      "id": 1,
4      "name": "Macbook Pro",
5      "description": "Ordinateur Apple",
6      "price": 1999.99,
7      "stock": 6,
8      "createdAt": "2026-03-20T18:42:38.372636",
9      "updatedAt": "2026-03-20T18:42:38.373633"
10   }
11 ]
```

Insérer ici le screenshot Postman de la requête `GET /api/products` — réponse `200 OK`.

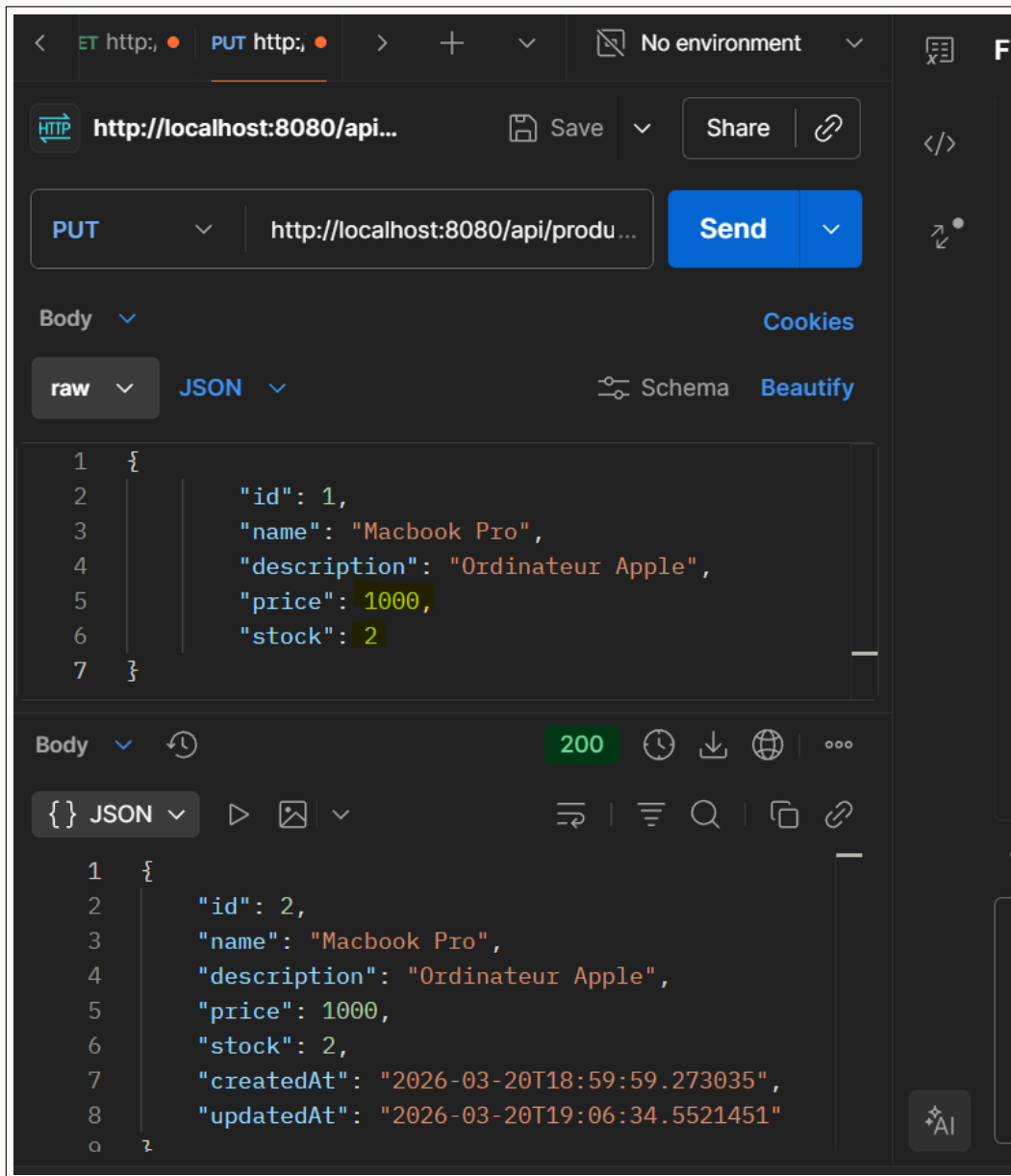
🔍 GET — Récupérer un produit par ID



Insérer ici le screenshot Postman de la requête `GET /api/products/{id}` — réponse `200 OK`.



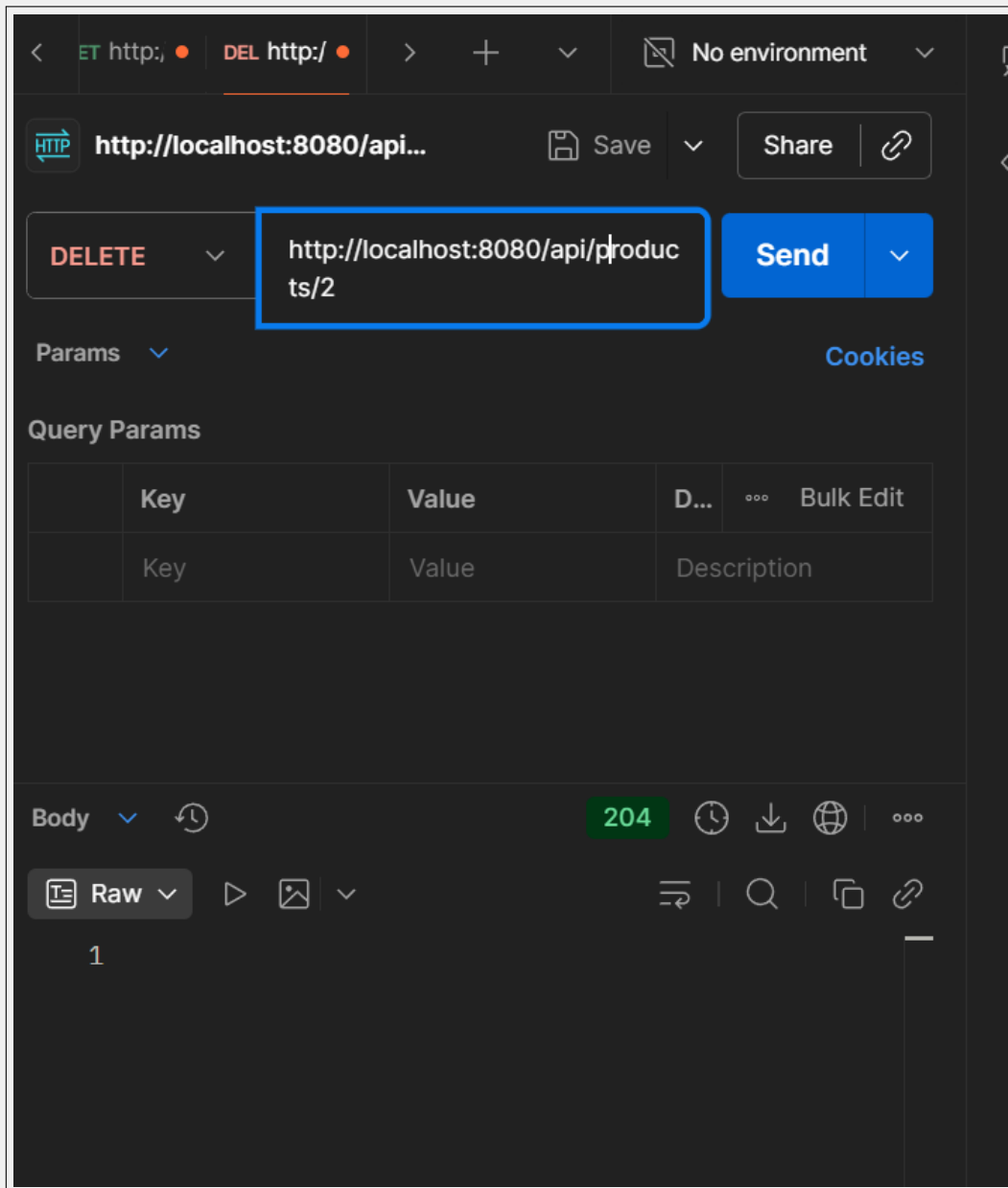
PUT — Modifier un produit



Insérer ici le screenshot Postman de la requête `PUT /api/products/{id}` — réponse 200 OK.

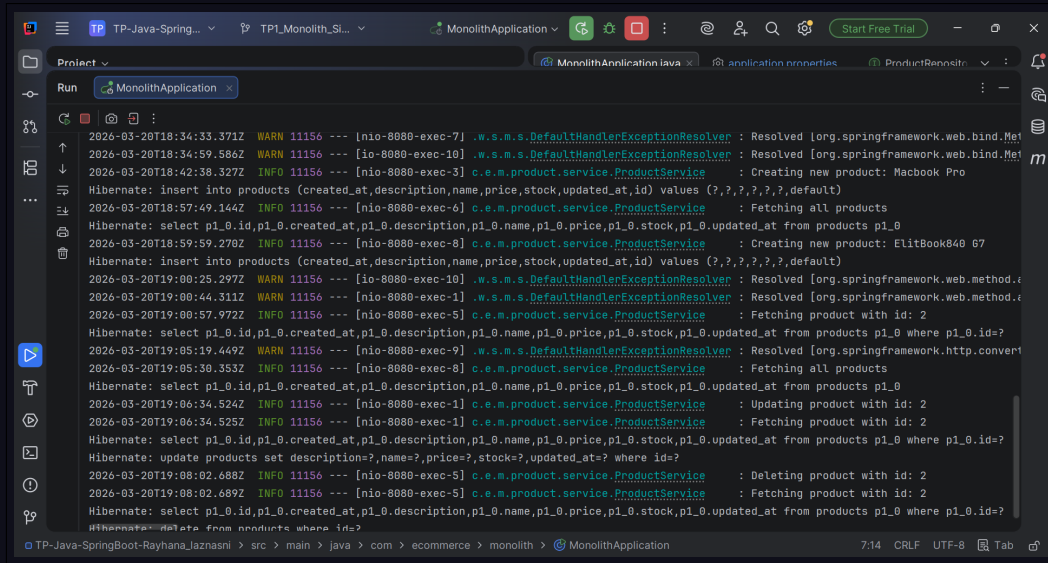


DELETE — Supprimer un produit



Insérer ici le screenshot Postman de la requête `DELETE /api/products/{id}` — réponse `204 No Content`.

>_ Terminal — Logs Spring Boot



```
2026-03-28T18:34:33.371Z WARN 11156 --- [nio-8080-exec-7] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.Me!
2026-03-28T18:34:59.586Z WARN 11156 --- [io-8080-exec-10] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.Me!
2026-03-28T18:42:38.327Z INFO 11156 --- [nio-8080-exec-3] c.e.m.product.service.ProductService : Creating new product: Macbook Pro
Hibernate: insert into products (created_at,description,name,price,stock,updated_at,id) values (?,?,?,?,?,default)
2026-03-28T18:57:49.144Z INFO 11156 --- [nio-8080-exec-6] c.e.m.product.service.ProductService : Fetching all products
Hibernate: select p1_0.id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock,p1_0.updated_at from products p1_0
2026-03-28T18:59:59.270Z INFO 11156 --- [nio-8080-exec-8] c.e.m.product.service.ProductService : Creating new product: ElitBook840 67
Hibernate: insert into products (created_at,description,name,price,stock,updated_at,id) values (?,?,?,?,?,default)
2026-03-28T19:00:25.297Z WARN 11156 --- [io-8080-exec-10] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.method.i
2026-03-28T19:00:44.311Z WARN 11156 --- [nio-8080-exec-1] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.method.i
2026-03-28T19:00:57.972Z INFO 11156 --- [nio-8080-exec-5] c.e.m.product.service.ProductService : Fetching product with id: 2
Hibernate: select p1_0.id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock,p1_0.updated_at from products p1_0 where p1_0.id=?
2026-03-28T19:05:19.449Z WARN 11156 --- [nio-8080-exec-9] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.http.convert
2026-03-28T19:05:38.353Z INFO 11156 --- [nio-8080-exec-8] c.e.m.product.service.ProductService : Fetching all products
Hibernate: select p1_0.id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock,p1_0.updated_at from products p1_0
2026-03-28T19:06:34.524Z INFO 11156 --- [nio-8080-exec-1] c.e.m.product.service.ProductService : Updating product with id: 2
2026-03-28T19:06:34.525Z INFO 11156 --- [nio-8080-exec-1] c.e.m.product.service.ProductService : Fetching product with id: 2
Hibernate: select p1_0.id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock,p1_0.updated_at from products p1_0 where p1_0.id=?
2026-03-28T19:08:02.688Z INFO 11156 --- [nio-8080-exec-5] c.e.m.product.service.ProductService : Deleting product with id: 2
2026-03-28T19:08:02.689Z INFO 11156 --- [nio-8080-exec-5] c.e.m.product.service.ProductService : Fetching product with id: 2
Hibernate: select p1_0.id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock,p1_0.updated_at from products p1_0 where p1_0.id=?
Hibernate: delete from products where id=?
```

Les logs Spring Boot (requêtes SQL générées par Hibernate).

6 Exercice — Entité Category



Exercice pratique

Nous avons ajouté une entité **Category** et créé la relation **@ManyToOne** avec **Product**. Nous avons testé avec **RestRepository**.

Travail demandé :

1. Créer la classe **Category** avec les champs **id** et **name**
2. Ajouter le champ **category** dans **Product** avec **@ManyToOne**
3. Créer **CategoryRepository** avec **@RepositoryRestResource**
4. Tester les endpoints générés automatiquement dans Postman

Indice :

Category.java — Squelette

```
1  @Entity
2  @Table(name = "categories")
3  @Getter @Setter @NoArgsConstructor @AllArgsConstructor
4  public class Category {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9
10     @NotBlank
11     @Column(nullable = false, unique = true)
12     private String name;
13 }
14
15 // Dans Product.java, ajouter :
16 @ManyToOne(fetch = FetchType.LAZY)
17 @JoinColumn(name = "category_id")
18 private Category category;
```

Code disponible sur GitHub — Branche TP2

Le code complet de l'exercice **Category** est disponible sur le dépôt GitHub dans la branche **tp2** :

```
 branch : tp2   
github.com/Neriona/Stakeholders-portfolio
```

 Cloner ou switcher sur la branche **tp2** : `git checkout tp2`

 Package : `com.ecommerce.monolith.category`

 Fichiers : `Category.java`, `CategoryRepository.java`, `Product.java` (mis à jour)

7 Analyse — Avantages et Limites du Monolithe

- ✓ Simple à développer et déployer
- ✓ Pas de latence réseau entre modules
- ✓ Débogage et tests plus faciles
- ✓ Transactions ACID natives
- ✓ Idéal pour les petits projets

- ✗ Scalabilité difficile
- ✗ Déploiement complet à chaque modification
- ✗ Couplage fort entre les modules
- ✗ Technologies figées pour tout le projet
- ✗ Risque de devenir ingérable à grande échelle

8 Conclusion

Ce TP nous a permis de construire une API REST complète avec Spring Boot en suivant l'architecture en couches **Entity** → **Repository** → **Service** → **Controller**.

Nous avons également découvert la puissance de `@RepositoryRestResource` qui nous permet de générer automatiquement des endpoints REST sans écrire de Controller, idéal pour le prototypage rapide.

➔ Notre prochaine étape sera de **décomposer** cette application monolithique en **microservices indépendants** pour bénéficier d'une meilleure scalabilité et flexibilité.

Réalisé par

Rayhana Laznasni

CSIA — Cybersécurité & Intelligence
Artificielle

ENSA BM · 2024/2025